Desafio Backend

Sistema simplificado de contas digitais

1. Contexto do desafio

Uma instituição financeira está desenvolvendo um sistema simplificado de contas digitais. O sistema precisa permitir que clientes realizem movimentações financeiras e consultar seus saldos e históricos de operações.

Durante os testes iniciais, a equipe identificou riscos importantes:

- Uma transferência pode debitar uma conta sem creditar a conta de destino.
- Uma requisição repetida pode executar a mesma transferência duas vezes.
- Duas transferências simultâneas podem gastar o mesmo saldo disponível.
- O histórico financeiro pode ficar inconsistente com os saldos das contas.

Você foi contratado como desenvolvedor backend para implementar uma API que resolva esses problemas.

Seu objetivo não é apenas fazer os endpoints funcionarem. É garantir a integridade financeira mesmo quando ocorrem falhas e requisições concorrentes.

2. Requisitos funcionais

RF01 — Criar conta

- O sistema deve permitir a abertura de uma conta digital.
- Cada conta deve possuir um identificador único.
- A conta deve iniciar com saldo zero.
- Deve possuir status, como ACTIVE ou BLOCKED.
- Uma conta bloqueada não pode realizar movimentações.

RF02 — Consultar conta e saldo

- O sistema deve permitir consultar os dados de uma conta e seu saldo atual.

RF03 — Realizar depósito

- O sistema deve permitir adicionar fundos a uma conta ativa.
- O valor deve ser positivo.
- A operação deve gerar um registro financeiro.
- A alteração do saldo e o registro do lançamento devem ser atômicos.

RF04 — Realizar saque

- O sistema deve permitir retirar fundos de uma conta ativa.
- O valor deve ser positivo.
- Não pode haver saldo negativo.
- A operação deve gerar um lançamento de débito.

RF05 — Realizar transferência

- O sistema deve permitir transferir dinheiro entre duas contas.
- A conta de origem e a conta de destino devem existir e estar ativas.
- As contas devem ser diferentes.
- O valor deve ser positivo.

- A conta de origem deve possuir saldo suficiente.
- O débito e o crédito devem ser concluídos juntos ou revertidos juntos.
- Cada transferência deve possuir um identificador único.

RF06 — Consultar extrato

- O sistema deve permitir consultar os lançamentos financeiros de uma conta, incluindo tipo, valor, data, identificação da operação e natureza do lançamento.

RF07 — Idempotência

- O sistema deve aceitar uma chave de idempotência nas operações financeiras que alteram o saldo.
- Se o cliente enviar a mesma requisição novamente com a mesma chave, a operação não poderá ser executada novamente.
- Se a mesma chave for reutilizada com parâmetros diferentes, a API deverá rejeitar a requisição.

3. Requisitos não funcionais

ID	Requisito
RNF01	Utilizar Java 21 e Spring Boot.
RNF02	Utilizar PostgreSQL como banco de dados.
RNF03	Utilizar BigDecimal para valores monetários.
RNF04	Garantir atomicidade das movimentações financeiras.
RNF05	Tratar concorrência e impedir gastos acima do saldo disponível.
RNF06	Implementar testes unitários e de integração.
RNF07	Disponibilizar ambiente executável via Docker Compose.
RNF08	Documentar a API com OpenAPI/Swagger.
RNF09	Registrar operações de forma rastreável, sem permitir a alteração arbitrária do histórico financeiro.

4. Regras de negócio obrigatórias

Estas regras são invariantes do sistema. Independentemente da arquitetura escolhida, elas precisam ser respeitadas.

- Uma conta não pode possuir saldo negativo.
- Um depósito aumenta o saldo exatamente pelo valor depositado.
- Um saque reduz o saldo exatamente pelo valor sacado.
- Uma transferência de valor V deve reduzir o saldo da origem em V e aumentar o saldo do destino em V.
- Uma transferência não pode ser executada parcialmente.
- Uma operação aceita não pode ser aplicada mais de uma vez por causa de uma repetição da requisição.
- O histórico financeiro não pode ser apagado ou editado para corrigir uma movimentação; correções devem ser registradas como novas operações compensatórias.
- O sistema deve manter a consistência mesmo quando duas ou mais operações são executadas simultaneamente.

Observação importante: para este desafio, considere todas as contas na mesma moeda, BRL. Não implemente juros, tarifas, câmbio, Pix real ou integração com bancos externos.

5. Contrato inicial da API

Os endpoints abaixo são uma proposta de contrato para orientar a implementação.

Método	Endpoint	Finalidade
POST	/api/v1/accounts	Criar conta
GET	/api/v1/accounts/{id}	Consultar conta e saldo
POST	/api/v1/accounts/{id}/deposits	Depositar
POST	/api/v1/accounts/{id}/withdrawals	Sacar
POST	/api/v1/transfers	Transferir
GET	/api/v1/accounts/{id}/ledger	Consultar extrato

Exemplo de requisição para transferência:

```
POST /api/v1/transfers
Content-Type: application/json
Idempotency-Key: 8f75d80a-2d71-4bb5-a8f0-13f9c21e4c00

{
  "sourceAccountId": "uuid-da-conta-origem",
  "destinationAccountId": "uuid-da-conta-destino",
  "amount": 150.00
}
```

Exemplo de resposta de sucesso:

```
{
  "transferId": "uuid-da-transferencia",
  "status": "COMPLETED",
  "amount": 150.00,
  "sourceAccountId": "uuid-da-conta-origem",
  "destinationAccountId": "uuid-da-conta-destino"
}
```

A resposta acima representa o contrato esperado; os identificadores são ilustrativos.

6. O principal teste: concorrência financeira

Considere o seguinte cenário:

- Conta A possui R$ 100,00.
- Conta B possui R$ 0,00.
- Conta C possui R$ 0,00.
- Duas requisições tentam transferir R$ 80,00 da conta A simultaneamente: uma para B e outra para C.

Estado inicial

Conta	Saldo
Conta A	R$ 100,00
Conta B	R$ 0,00
Conta C	R$ 0,00

Duas transferências simultâneas de R$ 80,00.

Resultado permitido

Uma transferência é concluída e a outra é rejeitada por saldo insuficiente. A ordem pode variar.

Resultado proibido

As duas transferências são concluídas, fazendo a conta A gastar R$ 160,00 apesar de possuir apenas R$ 100,00.

Você deve demonstrar esse cenário por meio de um teste de integração com requisições concorrentes contra PostgreSQL real, não apenas por testes unitários com mocks.